

HEC National Skill Competency Test (NSCT)

AI / Machine Learning & Data Analytics

Complete Study Notes | All 17 Topics

Comprehensive notes with explanations, examples & exam tips

Prepared for NSCT Excellence

Table of Contents

Topic #	Title
1	Introduction to AI, ML & Data Analytics
2	Mathematical Foundations
3	Python for AI & Data Analytics
4	Data Collection & Pre-processing
5	Exploratory Data Analysis (EDA)
6	Supervised Learning
7	Ensemble Learning
8	Unsupervised Learning
9	Model Evaluation & Validation
10	Feature Engineering & Selection
11	Deep Learning Fundamentals
12	Advanced Deep Learning
13	Natural Language Processing (NLP)

14	Computer Vision
15	Big Data Analytics (Introductory)
16	Model Deployment & MLOps Basics
17	AI Ethics, Security & Privacy

Topic 1: Introduction to AI, ML & Data Analytics

1.1 What is Artificial Intelligence (AI)?

Artificial Intelligence (AI) is the simulation of human intelligence processes by machines, especially computer systems. AI enables machines to perform tasks that normally require human intelligence such as visual perception, speech recognition, decision-making, and language translation.

Key Definition

AI = Science + Engineering of making intelligent machines that perceive their environment and take actions that maximize the chance of achieving their goals.

1.2 Types of AI

Type	Definition	Example
Narrow AI (Weak AI)	Designed for a specific task	Siri, Chess engines, Spam filters
General AI (Strong AI)	Human-level intelligence across all domains	Hypothetical – does not yet exist
Super AI	Surpasses human intelligence in all aspects	Theoretical future concept

1.3 What is Machine Learning (ML)?

Machine Learning is a subset of AI that gives systems the ability to automatically learn and improve from experience without being explicitly programmed. Instead of writing rules, you feed data to an algorithm, and the algorithm learns the rules itself.

Types of ML (overview — detailed in Topics 6-8):

- Supervised Learning – learns from labelled data (e.g., email spam detection)
- Unsupervised Learning – finds patterns in unlabelled data (e.g., customer clustering)
- Reinforcement Learning – learns by interacting with environment (e.g., game-playing AI)

1.4 What is Data Analytics?

Data Analytics is the process of examining data sets to draw conclusions and support decision-making. It uses statistics, programming, and domain knowledge to turn raw data into actionable insights.

Level	Question Answered	Technique
Descriptive Analytics	What happened?	Summary stats, dashboards
Diagnostic Analytics	Why did it happen?	Drill-down, correlations
Predictive Analytics	What will happen?	ML models, forecasting

Level	Question Answered	Technique
Prescriptive Analytics	What should we do?	Optimization, simulation

1.5 AI vs ML vs Data Science vs Data Analytics

These terms are related but distinct. Think of them as nested circles: AI is the broadest field. ML is a technique within AI. Data Science uses ML plus statistics. Data Analytics focuses on deriving insights from structured/unstructured data.

Field	Focus	Core Tools
Artificial Intelligence	Creating intelligent behaviour	Logic, Search, ML, NLP
Machine Learning	Learning from data	Algorithms, Statistics
Data Science	Extracting knowledge from data	Python, R, ML, Visualization
Data Analytics	Business insight from data	SQL, Excel, BI Tools, Stats

■ **EXAM TIP:** Know the hierarchy: AI ⊃ ML ⊃ Deep Learning. Data Science overlaps with all three. Exam often tests the difference between these fields.

Topic 2: Mathematical Foundations

2.1 Linear Algebra

Linear algebra is the backbone of machine learning. Most ML algorithms are expressed as matrix/vector operations.

Scalars, Vectors, Matrices, Tensors

Object	Definition	Example
Scalar	Single number	5, -3.14
Vector	1-D array of numbers	[2, 4, 6]
Matrix	2-D array (rows x cols)	[[1,2],[3,4]]
Tensor	n-D array ($n \geq 3$)	Image pixel data (H x W x C)

Key Operations:

- Matrix Multiplication: ($A \times B$) requires A's columns == B's rows
- Transpose: Flip rows and columns — A^T
- Dot Product: $u \cdot v = u_1v_1 + u_2v_2 + \dots$ measures similarity
- Eigenvalues / Eigenvectors: Used in PCA dimensionality reduction
- Norm ($\|v\|$): Measures length/magnitude of a vector

2.2 Calculus — Derivatives & Gradients

Calculus enables the optimisation of ML models. The derivative measures how a function changes as its input changes. Gradients are multi-variable extensions of derivatives.

- Derivative: df/dx — rate of change of f with respect to x
- Gradient: vector of partial derivatives ∇f — direction of steepest ascent
- Chain Rule: essential for backpropagation in neural networks
- Gradient Descent: update rule — $w = w - \alpha \cdot \nabla L(w)$, where α is learning rate

Example – Gradient Descent

Loss function: $L(w) = (y - wx)^2$

Gradient: $dL/dw = -2x(y - wx)$

Update: $w_{\text{new}} = w_{\text{old}} - \alpha \cdot (-2x(y - wx))$

This iteratively minimises the loss by moving w in the direction that reduces L .

2.3 Probability & Statistics

Probability is the mathematical framework for reasoning under uncertainty — critical for building robust ML models.

Core Concepts:

Concept	Formula / Description
Probability $P(A)$	$0 \leq P(A) \leq 1$
Joint $P(A \cap B)$	$P(A) \times P(B)$ if independent
Conditional $P(A B)$	$P(A \cap B) / P(B)$
Bayes' Theorem	$P(A B) = P(B A) \cdot P(A) / P(B)$
Expected Value $E[X]$	$\sum x \cdot P(x)$ for discrete X
Variance $\text{Var}(X)$	$E[(X - \mu)^2]$
Standard Deviation σ	$\sqrt{\text{Var}(X)}$

Key Probability Distributions:

Distribution	Use in ML	Parameters
Normal (Gaussian)	Assumed error distribution, weight initialisation	μ (mean), σ (std)
Bernoulli	Binary classification output	p (success probability)
Binomial	Count of successes in n trials	n, p
Uniform	Random initialisation	a (min), b (max)
Poisson	Count data modelling	λ (rate)

2.4 Statistics Essentials

- Mean: average value — $\mu = \sum x / n$
- Median: middle value — robust to outliers
- Mode: most frequent value
- Variance / Std Dev: spread of data
- Skewness: asymmetry of distribution (positive = right tail)
- Kurtosis: peakedness (excess kurtosis > 0 = heavy tails)
- Correlation: $r \in [-1, 1]$ — linear relationship between two variables
- Covariance: how two variables vary together

■ **EXAM TIP: Bayes' Theorem is tested frequently. Memorise: $P(A|B) = P(B|A) \cdot P(A) / P(B)$. Also know the difference between population parameters and sample statistics.**

Topic 3: Python for AI & Data Analytics

3.1 Why Python?

Python has become the dominant language for AI and Data Analytics due to its simplicity, readability, and a vast ecosystem of libraries. It allows rapid prototyping and has excellent community support.

- Simple, readable syntax — easy to learn
- Rich ecosystem: NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch
- Interactive environments: Jupyter Notebooks
- Strong community and open-source libraries

3.2 Key Libraries Overview

Library	Purpose	Key Functions
NumPy	Numerical computing / arrays	array(), reshape(), dot(), mean(), std()
Pandas	Data manipulation / DataFrames	read_csv(), groupby(), merge(), fillna()
Matplotlib	Basic 2D plotting	plot(), bar(), hist(), scatter()
Seaborn	Statistical visualisation	heatmap(), pairplot(), boxplot()
Scikit-learn	ML algorithms & pipelines	fit(), predict(), train_test_split()
TensorFlow	Deep learning framework	tf.keras, layers, Model, compile()
PyTorch	Deep learning (research-focused)	torch.tensor, nn.Module, autograd
NLTK / spaCy	Natural Language Processing	tokenize, POS tag, NER
OpenCV	Computer Vision	imread(), resize(), cvtColor()

3.3 NumPy Essentials

NumPy provides the ndarray — a fast, memory-efficient n-dimensional array:

```
import numpy as np
a = np.array([1,2,3,4,5]) # 1-D array
A = np.zeros((3,3)) # 3x3 matrix of zeros
B = np.eye(3) # 3x3 identity matrix
np.dot(A, B) # Matrix multiplication
a.reshape(5,1) # Change shape
```

3.4 Pandas Essentials

Pandas uses DataFrames (2D tables) and Series (1D arrays) for data manipulation:

```
import pandas as pd
df = pd.read_csv('data.csv') # Load data
df.head() # First 5 rows
df.describe() # Summary statistics
df['col'].fillna(df['col'].mean()) # Fill missing values
df.groupby('category')['sales'].sum() # Group and aggregate
df.merge(df2, on='id', how='inner') # Merge two DataFrames
```

3.5 Scikit-learn Workflow

Standard ML pipeline with scikit-learn:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
scaler = StandardScaler(); X_train = scaler.fit_transform(X_train)
model = LogisticRegression(); model.fit(X_train, y_train)
preds = model.predict(X_test)
print(accuracy_score(y_test, preds))
```

■ **EXAM TIP: Know the scikit-learn API: fit(), transform(), fit_transform(), predict(), predict_proba(). These appear in almost every ML question.**

Topic 4: Data Collection & Pre-processing

4.1 Data Collection Methods

Method	Description	Example
Primary Data	Collected directly by researcher	Surveys, experiments, sensors
Secondary Data	Already existing data	Kaggle, UCI, government open data
Web Scraping	Extract data from websites via code	BeautifulSoup, Scrapy
APIs	Access structured data from services	Twitter API, OpenWeatherMap
Databases	Structured data stored in RDBMS	SQL queries on MySQL/PostgreSQL
IoT / Sensors	Real-time physical world data	Temperature, GPS, accelerometers

4.2 Types of Data

- Structured: organised in rows/columns — CSV, SQL tables
- Semi-structured: JSON, XML — have tags but flexible schema
- Unstructured: text, images, audio, video — no predefined format
- Time-series: data indexed by time — stock prices, sensor readings

4.3 Data Quality Issues

Issue	Description	Solution
Missing Values	NaN / null entries in dataset	Imputation, drop rows/cols
Outliers	Extreme values far from distribution	IQR method, Z-score, cap/floor
Duplicates	Same record appears multiple times	<code>df.drop_duplicates()</code>
Inconsistency	Same entity stored differently	Standardise formats
Noise	Random errors in measurements	Smoothing, filtering
Imbalance	Unequal class distribution	SMOTE, oversampling

4.4 Handling Missing Values

Missing data strategies depend on the nature and amount of missing data:

- Drop rows: when missing data < 5% and missing at random
- Mean/Median imputation: numerical features — median for skewed data
- Mode imputation: categorical features
- Forward Fill / Backward Fill: for time-series data
- KNN Imputation: use similar records to fill values
- Model-based imputation: predict missing values using ML

4.5 Data Encoding

Most ML algorithms require numerical input. Categorical variables must be encoded:

Method	Use Case	Example
Label Encoding	Ordinal categories (have order)	low=0, medium=1, high=2
One-Hot Encoding	Nominal categories (no order)	Red=[1,0,0], Blue=[0,1,0]
Binary Encoding	High-cardinality categories	100 → [1,0,0]
Target Encoding	Replace category with target mean	City → avg_salary

4.6 Feature Scaling

Many algorithms (SVM, KNN, Neural Networks) are sensitive to feature magnitudes. Feature scaling ensures all features contribute equally.

Method	Formula	When to Use
Min-Max Normalization	$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$	When data has known bounds [0,1]
Standardization (Z-score)	$x' = (x - \mu) / \sigma$	When data is approx. Gaussian
Robust Scaler	Uses median and IQR	When data has outliers

■ **EXAM TIP: Remember: Decision Trees and Random Forests do NOT require feature scaling. SVM, KNN, Logistic Regression, and Neural Networks DO require it.**

Topic 5: Exploratory Data Analysis (EDA)

5.1 What is EDA?

EDA is the process of analysing data sets to summarise their main characteristics, often using visual methods. It was pioneered by statistician John Tukey. EDA is done BEFORE modelling to understand the data, detect anomalies, test assumptions, and discover patterns.

5.2 EDA Steps

- Step 1: Understand the data — shape, types, head/tail
- Step 2: Summary statistics — mean, median, std, min, max
- Step 3: Check missing values and duplicates
- Step 4: Univariate analysis — distribution of each feature
- Step 5: Bivariate analysis — relationship between pairs of features
- Step 6: Multivariate analysis — interactions among multiple features
- Step 7: Outlier detection
- Step 8: Correlation analysis

5.3 Key Visualisations

Chart Type	Use	Python
Histogram	Distribution of numerical variable	<code>plt.hist()</code> / <code>sns.histplot()</code>
Box Plot	Spread, median, outliers	<code>sns.boxplot()</code>
Bar Chart	Frequency of categories	<code>plt.bar()</code> / <code>sns.barplot()</code>
Scatter Plot	Relationship between 2 numerical vars	<code>plt.scatter()</code> / <code>sns.scatterplot()</code>
Heatmap	Correlation matrix visualisation	<code>sns.heatmap(df.corr())</code>
Pair Plot	All pairwise scatter plots	<code>sns.pairplot(df)</code>
Violin Plot	Distribution + probability density	<code>sns.violinplot()</code>
Count Plot	Count of categorical values	<code>sns.countplot()</code>
Line Plot	Trends over time	<code>plt.plot()</code>

5.4 Correlation Analysis

Pearson correlation coefficient (r) measures linear relationship between two continuous variables. Values range from -1 to +1.

r value	Interpretation
$r = 1.0$	Perfect positive correlation
$0.7 \leq r < 1.0$	Strong positive correlation
$0.3 \leq r < 0.7$	Moderate positive correlation
$r \approx 0$	No linear correlation
$-0.7 \leq r < -0.3$	Moderate negative correlation
$r = -1.0$	Perfect negative correlation

5.5 Outlier Detection Methods

- IQR Method: Outlier if $x < Q1 - 1.5 \cdot IQR$ OR $x > Q3 + 1.5 \cdot IQR$
- Z-Score Method: Outlier if $|z| > 3$, where $z = (x - \mu) / \sigma$
- Visual: Box plots clearly show outliers as points beyond whiskers

■ **EXAM TIP: EDA is often tested through interpretation questions. Given a histogram, identify skewness (right skew = positive, left skew = negative). Given a correlation heatmap, identify multicollinearity.**

Topic 6: Supervised Learning

6.1 Definition

Supervised Learning is the task of learning a mapping function $f: X \rightarrow Y$ from labelled training data, where X is the input and Y is the output/label. The algorithm 'learns' by comparing its predictions to the true labels and minimising the error.

- Classification: Y is discrete (e.g., spam/not-spam, cat/dog)
- Regression: Y is continuous (e.g., house price, temperature)

6.2 Linear Regression

Models the relationship between a dependent variable Y and one or more independent variables X using a linear equation.

Simple Linear Regression: $Y = \beta_0 + \beta_1 \cdot X + \epsilon$

Multiple Linear Regression: $Y = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \dots + \beta_n \cdot X_n + \epsilon$

β_0 = intercept (bias), $\beta_1 \dots \beta_n$ = coefficients (weights), ϵ = error term. Parameters are estimated by minimising MSE using Ordinary Least Squares (OLS) or Gradient Descent.

6.3 Logistic Regression

Despite the name, Logistic Regression is a CLASSIFICATION algorithm. It uses the sigmoid function to output a probability between 0 and 1.

$\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$ where $z = \beta_0 + \beta_1 \cdot X_1 + \dots + \beta_n \cdot X_n$

Loss function: Binary Cross-Entropy = $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$

6.4 Decision Trees

A Decision Tree splits data recursively based on feature values, creating a tree-like structure. Each node is a decision rule, each leaf is a prediction.

- Splitting criterion (Classification): Gini Impurity or Information Gain (Entropy)
- Splitting criterion (Regression): Mean Squared Error (MSE)
- Entropy: $H = -\sum p_i \cdot \log_2(p_i)$ — measures impurity
- Gini: $G = 1 - \sum p_i^2$ — computational efficiency advantage
- Advantages: interpretable, no feature scaling needed, handles non-linear data
- Disadvantages: prone to overfitting, unstable (high variance)

6.5 Support Vector Machine (SVM)

SVM finds the optimal hyperplane that maximises the margin between two classes. Support vectors are data points closest to the hyperplane.

- Hard Margin SVM: assumes linearly separable data
- Soft Margin SVM: allows some misclassification (C parameter controls trade-off)

- Kernel Trick: transforms data to higher dimension — RBF, Polynomial, Linear kernels
- Large C = low bias, high variance (smaller margin, fewer errors)
- Small C = high bias, low variance (larger margin, more errors allowed)

6.6 K-Nearest Neighbours (KNN)

KNN is a non-parametric, instance-based (lazy) learning algorithm. Classification: majority vote of K nearest neighbours. Regression: mean of K nearest neighbours.

- Distance metrics: Euclidean = $\sqrt{\sum (x_i - y_i)^2}$, Manhattan = $\sum |x_i - y_i|$
- Small K: complex decision boundary, overfitting
- Large K: smooth decision boundary, underfitting
- Requires feature scaling — sensitive to magnitudes

6.7 Naive Bayes

Uses Bayes' Theorem with the 'naive' assumption that all features are conditionally independent given the class.

$$P(\text{class} \mid \text{features}) \propto P(\text{class}) \cdot \prod P(\text{feature}_i \mid \text{class})$$

- Gaussian Naive Bayes: continuous features assumed to be normally distributed
- Multinomial Naive Bayes: count data (text classification)
- Very fast, works well for text classification (spam detection)

■ **EXAM TIP: For classification, memorise: Logistic Regression (linear boundary), SVM (max margin), KNN (distance-based), Decision Tree (rule-based), Naive Bayes (probabilistic). Know when to use each.**

Topic 7: Ensemble Learning

7.1 What is Ensemble Learning?

Ensemble learning combines multiple individual models (weak learners) to produce a stronger, more accurate model. The idea: the collective wisdom of many models outperforms any single model. Analogous to asking many experts and combining opinions.

7.2 Bagging (Bootstrap Aggregating)

Trains multiple models in PARALLEL on different random subsets of the training data (with replacement). Reduces VARIANCE (overfitting).

Random Forest — the most famous bagging algorithm:

- Builds many decision trees on bootstrap samples
- Each tree sees a random subset of features at each split (feature randomness)
- Classification: majority vote across all trees
- Regression: average prediction across all trees
- Key hyperparameters: `n_estimators` (number of trees), `max_depth`, `max_features`
- Feature Importance: can rank features by how much they reduce impurity

7.3 Boosting

Trains models SEQUENTIALLY, where each new model focuses on the errors of the previous model. Reduces BIAS. Stronger than bagging in many tasks.

AdaBoost (Adaptive Boosting):

- Assigns weights to training samples — misclassified samples get higher weight
- Each weak learner (stump) is weighted by its accuracy
- Final prediction: weighted sum of all weak learner predictions

Gradient Boosting:

- Each new tree fits the RESIDUALS (errors) of the previous ensemble
- Gradient descent in function space — minimises a loss function
- XGBoost: optimised, regularised implementation — wins many Kaggle competitions
- LightGBM: faster training, leaf-wise tree growth — great for large datasets
- CatBoost: handles categorical features natively

7.4 Stacking

Trains multiple diverse base models (Level-0 learners) and uses a meta-learner (Level-1 learner) that takes their predictions as inputs to produce the final output.

- Base learners: Random Forest, SVM, KNN, Logistic Regression
- Meta-learner: Logistic Regression or Linear Regression

- Avoids overfitting via cross-validation when generating meta-features

Method	Training	Parallel/Sequential	Reduces	Key Algorithm
Bagging	Bootstrap subsets	Parallel	Variance	Random Forest
Boosting	Focus on errors	Sequential	Bias	XGBoost, AdaBoost
Stacking	Full dataset	Both	Both	Meta-learner

■ **EXAM TIP:** Exam often asks: 'What does bagging reduce?' → **Variance**. 'What does boosting reduce?' → **Bias**. Know the difference between AdaBoost (reweights samples) and Gradient Boosting (fits residuals).

Topic 8: Unsupervised Learning

8.1 Definition

Unsupervised Learning finds hidden patterns or structure in UNLABELLED data — no target variable Y is provided. The algorithm discovers the inherent organisation in the data.

8.2 Clustering

Clustering groups data points so that similar points are in the same group (cluster) and dissimilar points are in different groups.

K-Means Clustering:

- Algorithm: (1) Choose K centroids randomly. (2) Assign each point to nearest centroid. (3) Update centroids to mean of assigned points. (4) Repeat until convergence.
- Distance: Euclidean distance to centroids
- Inertia (WCSS): within-cluster sum of squares — minimised by K-Means
- Elbow Method: plot WCSS vs K — choose K at the 'elbow' inflection point
- Limitation: K must be specified upfront; sensitive to outliers; assumes spherical clusters

DBSCAN (Density-Based Spatial Clustering):

- Groups points in dense regions, marks low-density points as noise/outliers
- Parameters: epsilon (neighbourhood radius), min_samples (minimum points to form cluster)
- Does NOT require specifying K; can find arbitrary-shaped clusters
- Core Point: has $\geq$ min_samples neighbours within epsilon radius

Hierarchical Clustering:

- Agglomerative (bottom-up): starts with each point as cluster, merges closest
- Divisive (top-down): starts with all in one cluster, splits recursively
- Dendrogram: tree diagram showing merge history — cut at desired level for K clusters
- Linkage methods: Single, Complete, Average, Ward

8.3 Dimensionality Reduction

Reduces the number of features while preserving important information. Combats the curse of dimensionality and aids visualisation.

Principal Component Analysis (PCA):

- Finds orthogonal axes (principal components) of maximum variance
- PC1: direction of maximum variance; PC2: second maximum, orthogonal to PC1
- Steps: (1) Standardise data, (2) Compute covariance matrix, (3) Find eigenvectors/eigenvalues, (4) Project data onto top K eigenvectors
- Explained Variance Ratio: choose K components that explain 90-95% of variance

t-SNE (t-Distributed Stochastic Neighbour Embedding):

- Non-linear dimensionality reduction — excellent for 2D/3D visualisation
- Preserves local structure (nearby points stay nearby)
- NOT used for data compression — only for visualisation

8.4 Association Rule Learning

Discovers interesting relationships between variables in large datasets.

- Support: $P(A \cap B)$ — how often the itemset appears
- Confidence: $P(B|A) = P(A \cap B) / P(A)$ — how often rule is correct
- Lift: $\text{Confidence} / P(B)$ — how much more likely B is given A (lift > 1 = positive association)
- Apriori Algorithm: generates candidate itemsets using the Apriori property (all subsets of frequent itemset must be frequent)
- Example: {bread, butter} → {jam}, Support=0.4, Confidence=0.8, Lift=1.6

■ **EXAM TIP: Know K-Means vs DBSCAN: K-Means requires K upfront, assumes spherical clusters. DBSCAN discovers K automatically, handles arbitrary shapes, detects outliers. PCA is linear; t-SNE is non-linear (visualisation only).**

Topic 9: Model Evaluation & Validation

9.1 Train / Validation / Test Split

- Training Set (60-70%): used to fit model parameters
- Validation Set (10-15%): used to tune hyperparameters
- Test Set (15-20%): held out — final unbiased evaluation

9.2 Cross-Validation

Cross-validation gives a more reliable performance estimate by using all data for both training and testing (no wasted data).

- K-Fold CV: split data into K folds; train on K-1, test on 1; repeat K times; average results
- Stratified K-Fold: preserves class proportions in each fold — important for imbalanced data
- Leave-One-Out (LOO): $K = n$ — maximum data use, computationally expensive

9.3 Classification Metrics

Confusion Matrix components for binary classification:

Metric	Formula	Meaning
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall correct predictions
Precision	$TP / (TP + FP)$	Of all predicted positive, how many are actually positive
Recall (Sensitivity)	$TP / (TP + FN)$	Of all actual positive, how many were correctly predicted
Specificity	$TN / (TN + FP)$	Of all actual negative, how many correctly predicted
F1 Score	$2 \cdot (P \cdot R) / (P+R)$	Harmonic mean of Precision and Recall
AUC-ROC	Area under ROC Curve	Overall discriminative ability (1.0 = perfect)

When to use which metric?

Accuracy: balanced classes with equal cost of errors

Precision: cost of false positive is high (e.g., spam detection — don't delete real emails)

Recall: cost of false negative is high (e.g., cancer detection — don't miss sick patients)

F1 Score: imbalanced classes where both precision and recall matter

AUC-ROC: comparing models regardless of threshold; good for imbalanced datasets

9.4 Regression Metrics

Metric	Formula	Notes
MAE	$\sum y - \hat{y} / n$	Robust to outliers; same units as y
MSE	$\sum (y - \hat{y})^2 / n$	Penalises large errors heavily
RMSE	$\sqrt{\text{MSE}}$	Same units as y; most interpretable
R ² Score	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	1=perfect fit, 0=baseline, <0=worse than mean

9.5 Bias-Variance Tradeoff

Total Error = Bias² + Variance + Irreducible Noise. Finding the right model complexity balances bias and variance.

Concept	Definition	Symptom	Solution
High Bias	Model too simple, misses patterns	Underfitting	More complex model, more features
High Variance	Model too complex, learns noise	Overfitting	Regularisation, more data, simpler model

9.6 Regularisation

- L1 Regularisation (Lasso): adds $|w|$ penalty — produces sparse models (feature selection)
- L2 Regularisation (Ridge): adds w^2 penalty — shrinks weights, prevents large coefficients
- ElasticNet: combines L1 + L2
- Dropout (Neural Networks): randomly drops neurons during training — reduces overfitting

■ **EXAM TIP: Memorise the Bias-Variance Tradeoff. Overfitting = high variance = solution is regularisation/more data. Underfitting = high bias = solution is more complex model.**

Topic 10: Feature Engineering & Selection

10.1 Feature Engineering

Feature Engineering is the process of using domain knowledge to create, transform, or extract features from raw data to improve ML model performance. It is often the most impactful step in the ML pipeline.

Common Feature Engineering Techniques:

- Polynomial Features: $x_1, x_2 \rightarrow x_1^2, x_2^2, x_1 \cdot x_2$ (captures non-linear relationships)
- Log Transformation: $\log(x)$ — reduces skewness in right-skewed features
- Binning/Discretisation: $\text{age} \rightarrow \text{age_group}$ (child/teen/adult/senior)
- Date/Time Features: extract year, month, day, hour, day_of_week, is_weekend
- Interaction Features: $\text{feature_A} \times \text{feature_B}$
- Aggregation Features: `groupby + mean/sum/std` for grouped data
- Encoding (see Topic 4): one-hot, label, target encoding

10.2 Feature Selection

Feature selection removes irrelevant or redundant features, reducing dimensionality, training time, and overfitting while often improving accuracy.

Filter Methods (model-independent, based on statistics):

- Correlation Coefficient: remove features highly correlated with other features
- Chi-Square Test: for categorical target — tests independence between feature and target
- ANOVA F-Test: for continuous features vs categorical target
- Mutual Information: measures dependency between feature and target

Wrapper Methods (use model performance to evaluate subsets):

- Forward Selection: start with no features, add one at a time (best improvement)
- Backward Elimination: start with all features, remove one at a time (least useful)
- Recursive Feature Elimination (RFE): train model, rank features by importance, remove least important

Embedded Methods (feature selection built into model training):

- Lasso (L1): drives unimportant feature weights to exactly zero
- Random Forest Feature Importance: average impurity reduction across all trees
- XGBoost Feature Importance: based on how often feature is used for splitting

10.3 Dimensionality Reduction vs Feature Selection

Aspect	Feature Selection	Dimensionality Reduction (PCA)
Interpretability	Retains original features	Creates new abstract components

Aspect	Feature Selection	Dimensionality Reduction (PCA)
Method	Subset of original features	Linear combination of features
Algorithms	RFE, Lasso, Chi-Square	PCA, t-SNE, Autoencoders
Use Case	When interpretability matters	When high-dimensional data (images, text)

■ **EXAM TIP:** Know the three categories of Feature Selection: Filter (statistics), Wrapper (model performance), Embedded (built into model). Random Forest and XGBoost have built-in feature importance — a key advantage.

Topic 11: Deep Learning Fundamentals

11.1 Artificial Neural Networks (ANN)

Deep Learning is a subset of ML using Artificial Neural Networks (ANNs) with multiple hidden layers. Inspired by the biological neural network in the human brain.

Perceptron (single neuron):

```
output = activation_function( $\sum(w_i \cdot x_i) + \text{bias}$ )
```

- Inputs (x): features
- Weights (w): learned parameters
- Bias (b): allows shifting the activation function
- Activation Function: introduces non-linearity

11.2 Activation Functions

Function	Formula	Range	Use / Notes
Sigmoid	$1/(1+e^{(-x)})$	[0,1]	Output layer for binary classification; vanishing gradient problem
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	[-1,1]	Hidden layers; zero-centred; still has vanishing gradient
ReLU	$\max(0, x)$	$[0, \infty)$	Most popular hidden layer activation; fast; dead neuron problem
Leaky ReLU	$\max(0.01x, x)$	$(-\infty, \infty)$	Fixes dead neuron problem of ReLU
Softmax	$e^{x_i} / \sum e^{x_j}$	[0,1], sum=1	Output layer for multi-class classification

11.3 Feedforward Network Architecture

- Input Layer: one neuron per feature
- Hidden Layers: learn progressively abstract representations
- Output Layer: one neuron (regression), one neuron with sigmoid (binary), K neurons with softmax (K-class)
- Deep Network: 2+ hidden layers

11.4 Backpropagation & Training

Backpropagation computes gradients of the loss function with respect to each weight using the chain rule. These gradients are used by gradient descent to update weights.

- Forward Pass: compute output and loss
- Backward Pass: compute gradients layer by layer using chain rule
- Weight Update: $w = w - \alpha \cdot (\partial L / \partial w)$ where α = learning rate

11.5 Optimisers

Optimiser	Description	Key Feature
SGD	Gradient descent on mini-batches	Noisy but escapes local minima
Momentum	SGD + exponential moving average of gradient	Dampens oscillations
RMSprop	Adapts learning rate per parameter	Good for RNNs
Adam	Combines Momentum + RMSprop	Default choice for most problems
AdaGrad	Larger updates for infrequent parameters	Good for sparse features (NLP)

11.6 Hyperparameters

- Learning Rate (α): most critical — too high = diverge; too low = slow convergence
- Batch Size: mini-batch typically 32, 64, 128
- Number of Epochs: passes over entire training set
- Number of Layers / Neurons: architecture design
- Dropout Rate: fraction of neurons dropped — typically 0.2-0.5
- Weight Initialisation: Xavier (tanh), He (ReLU) initialisation strategies

■ **EXAM TIP: Adam optimiser is the default choice in most modern deep learning. ReLU is the default activation for hidden layers. Softmax for multi-class output, Sigmoid for binary classification output.**

Topic 12: Advanced Deep Learning

12.1 Convolutional Neural Networks (CNNs)

CNNs are specialised for grid-structured data (images, time-series). They learn spatial hierarchies of features through convolutional filters.

CNN Architecture: $\text{Input} \rightarrow [\text{Conv} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Pool}]^* \rightarrow \text{Flatten} \rightarrow \text{FC} \rightarrow \text{Output}$

- Convolutional Layer: applies learnable filters (kernels) — detects edges, textures, shapes
- Filter/Kernel: small matrix (e.g., 3x3) slid over input — produces feature map
- Stride: how many pixels filter moves at each step
- Padding (same/valid): same = preserve spatial dimensions; valid = reduce dimensions
- Pooling Layer: Max Pooling or Average Pooling — reduces spatial dimensions, translation invariance
- Batch Normalisation: normalises layer activations — speeds training, reduces sensitivity to init
- Fully Connected Layers: classification head after feature extraction

Famous CNN Architectures:

Architecture	Year	Key Innovation	Depth
LeNet-5	1998	First practical CNN for digit recognition	7 layers
AlexNet	2012	Deep CNN, ReLU, Dropout, GPU training	8 layers
VGGNet	2014	Very deep with small 3x3 filters	16-19 layers
ResNet	2015	Skip connections (residual learning)	50-152 layers
Inception	2014	Inception modules, multiple filter sizes	22 layers
EfficientNet	2019	Compound scaling of depth/width/resolution	Varies

12.2 Recurrent Neural Networks (RNNs)

RNNs process sequential data by maintaining a hidden state that captures information from previous time steps.

- Hidden state: $h_t = \tanh(W_h \cdot h_{(t-1)} + W_x \cdot x_t + b)$
- Vanishing Gradient Problem: gradients shrink exponentially through time — RNN forgets long-term dependencies
- LSTM (Long Short-Term Memory): solves vanishing gradient with gates

- GRU (Gated Recurrent Unit): simplified LSTM, fewer parameters, similar performance

LSTM Gates:

- Forget Gate: decides what to discard from cell state
- Input Gate: decides what new information to store
- Output Gate: decides what to output from cell state
- Cell State: long-term memory — information highway

12.3 Transformer Architecture

Transformers replaced RNNs as the dominant architecture for sequence tasks. They use self-attention mechanisms to relate positions within a sequence in parallel.

- Self-Attention: each token attends to all other tokens — captures global dependencies
- Multi-Head Attention: multiple attention heads learn different relationships
- Positional Encoding: adds position information (since no inherent sequence processing)
- Encoder-Decoder: Encoder processes input; Decoder generates output
- BERT: encoder-only, bidirectional, pre-trained on masked language modelling
- GPT: decoder-only, autoregressive, pre-trained on next-token prediction

12.4 Transfer Learning & Fine-Tuning

Transfer Learning uses a model pre-trained on a large dataset as a starting point for a new (often smaller) dataset and task.

- Feature Extraction: freeze pre-trained weights, add new classification head
- Fine-Tuning: unfreeze some/all layers and train with small learning rate
- Examples: ResNet pre-trained on ImageNet for medical imaging; BERT for sentiment analysis

■ **EXAM TIP: ResNet introduced skip connections to solve the vanishing gradient problem in very deep networks. LSTM solved the vanishing gradient in RNNs. Transformers use Attention, not recurrence — key difference from RNNs.**

Topic 13: Natural Language Processing (NLP)

13.1 NLP Pipeline

Standard NLP preprocessing pipeline:

- Step 1: Raw Text
- Step 2: Tokenisation
- Step 3: Lowercasing
- Step 4: Stop Word Removal
- Step 5: Stemming / Lemmatisation
- Step 6: POS Tagging
- Step 7: NER / Parsing
- Step 8: Feature Extraction
- Step 9: Model

13.2 Text Preprocessing

Technique	Description	Example
Tokenisation	Split text into tokens (words/sentences)	'Hello world' → ['Hello','world']
Lowercasing	Convert to lowercase	'NLP' → 'nlp'
Stop Word Removal	Remove common words with little meaning	Remove: 'the','is','a','of'
Stemming	Reduce to root form (crude)	'running','runs' → 'run'
Lemmatisation	Reduce to dictionary base form (smart)	'better' → 'good'
POS Tagging	Label grammatical role	'runs'→VERB, 'cat'→NOUN
Named Entity Recog.	Extract entities (person, org, location)	'Apple' → ORG

13.3 Text Representation (Feature Extraction)

Bag of Words (BoW):

- Represents text as vector of word counts — ignores word order and semantics
- Vocabulary: all unique words across corpus
- Sparse, high-dimensional representation

TF-IDF (Term Frequency – Inverse Document Frequency):

$$TF(t,d) = \text{count}(t \text{ in } d) / \text{total_words}(d)$$

$$IDF(t) = \log(N / df(t)) \text{ where } N=\text{docs}, df=\text{docs containing term}$$

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

- Words frequent in document but rare in corpus get HIGH score
- Stop words get low IDF → low TF-IDF score

Word Embeddings:

- Word2Vec: neural network maps words to dense vectors — captures semantic similarity
- GloVe: uses global co-occurrence statistics for word vectors
- FastText: character-level embeddings — handles out-of-vocabulary words
- BERT Embeddings: contextual — 'bank' has different vector in different contexts

13.4 NLP Tasks and Models

Task	Description	Models
Text Classification	Assign label to text	Naive Bayes, BERT, LSTM
Sentiment Analysis	Positive/Negative/Neutral	VADER, BERT, LSTM
Named Entity Recognition	Extract entities	spaCy, BERT-NER
Machine Translation	Translate languages	Transformer (seq2seq)
Text Summarisation	Shorten text	T5, BART, GPT
Question Answering	Answer from context	BERT, RoBERTa
Language Generation	Generate coherent text	GPT-2, GPT-4, LLaMA

■ **EXAM TIP: TF-IDF vs BoW: TF-IDF is better because it downweights common words. Word2Vec vs BERT: Word2Vec gives static embeddings; BERT gives contextual embeddings. BERT = bidirectional Transformer encoder.**

Topic 14: Computer Vision

14.1 What is Computer Vision?

Computer Vision enables computers to interpret and understand visual information from the world (images and videos). It combines deep learning with image processing techniques.

14.2 Image Representation

- Grayscale Image: 2D array of pixel intensities — shape (H, W)
- Colour Image (RGB): 3D array — shape (H, W, 3) for Red, Green, Blue channels
- Pixel values: 0 (black) to 255 (white) for 8-bit images
- Normalise: divide by 255 to scale pixel values to [0,1]

14.3 Image Preprocessing

- Resizing: standardise input dimensions for CNN (e.g., 224×224)
- Normalisation: scale pixel values to [0,1] or [-1,1]
- Colour Space Conversion: RGB → Grayscale, RGB → HSV
- Noise Reduction: Gaussian Blur, Median Filter
- Edge Detection: Sobel, Canny operators

14.4 Data Augmentation

Data augmentation artificially expands training dataset by applying random transformations. Critical to prevent overfitting when data is limited.

- Horizontal/Vertical Flip
- Rotation (e.g., ±15°)
- Zoom In/Out
- Brightness / Contrast adjustment
- Random Cropping
- Cutout / Mixup (advanced techniques)

14.5 Core Computer Vision Tasks

Task	Description	Key Models
Image Classification	Assign one label to entire image	ResNet, VGG, EfficientNet
Object Detection	Locate + classify multiple objects	YOLO, SSD, Faster R-CNN
Image Segmentation	Label every pixel	U-Net, DeepLab, Mask R-CNN
Face Recognition	Identify faces	FaceNet, ArcFace
Image Generation	Create synthetic images	GAN, Stable Diffusion

Task	Description	Key Models
Pose Estimation	Detect body keypoints	OpenPose, MediaPipe
OCR	Extract text from images	Tesseract, EasyOCR

14.6 Object Detection Deep Dive

- YOLO (You Only Look Once): single-pass detection — extremely fast, real-time capable
- Faster R-CNN: two-stage detector — Region Proposal Network + classification head, more accurate
- SSD (Single Shot Detector): multi-scale detection, balance of speed and accuracy
- IoU (Intersection over Union): measures overlap between predicted and ground truth box
- $\text{IoU} \geq 0.5$: typically considered a correct detection
- NMS (Non-Maximum Suppression): removes duplicate bounding boxes

■ **EXAM TIP: YOLO = speed (real-time), Faster R-CNN = accuracy. Segmentation gives pixel-level labels (U-Net famous for medical imaging). IoU measures detection quality.**

Topic 15: Big Data Analytics (Introductory)

15.1 What is Big Data?

Big Data refers to datasets that are too large and complex for traditional data processing tools. Defined by the 5 V's framework.

V	Definition	Example
Volume	Size of data	Petabytes/Exabytes of data
Velocity	Speed of data generation	Millions of tweets per minute
Variety	Different types of data	Text, images, structured, sensor data
Veracity	Uncertainty/quality of data	Social media noise, biased samples
Value	Useful insights extractable	Business decisions from analysis

15.2 Big Data Ecosystem

Tool / Technology	Purpose	Key Feature
Hadoop	Distributed storage & processing	HDFS (storage) + MapReduce (compute)
Apache Spark	Fast in-memory distributed compute	10-100x faster than Hadoop MapReduce
HDFS	Hadoop Distributed File System	Stores data across cluster nodes
Hive	SQL on Hadoop	HiveQL — query big data with SQL
HBase	NoSQL on Hadoop	Real-time read/write access
Kafka	Real-time streaming pipeline	High-throughput message queue
Spark Streaming	Real-time stream processing	Micro-batch or continuous processing
Flink	True streaming analytics	Low-latency event processing

15.3 MapReduce Paradigm

MapReduce splits computation into two phases — Map (parallel processing) and Reduce (aggregation) — across a cluster of computers.

- Map Phase: apply function to each record independently and emit (key, value) pairs

- Shuffle & Sort: group all values by key across the cluster
- Reduce Phase: aggregate values for each key

Example — Word Count:

Input: 'the cat sat the cat'

Map: ('the',1), ('cat',1), ('sat',1), ('the',1), ('cat',1)

Reduce: ('cat',2), ('sat',1), ('the',2)

15.4 Batch vs Stream Processing

Aspect	Batch Processing	Stream Processing
Data	Stored, historical data	Real-time, continuous data
Latency	High (hours/days)	Low (milliseconds/seconds)
Tools	Hadoop MapReduce, Hive	Kafka, Spark Streaming, Flink
Use Case	Monthly reports, ETL pipelines	Fraud detection, live dashboards

■ **EXAM TIP: Spark vs Hadoop: Spark processes in memory — 100x faster. Know the 5 V's and be able to explain each. MapReduce: Map = parallel process; Reduce = aggregate.**

Topic 16: Model Deployment & MLOps Basics

16.1 What is MLOps?

MLOps (Machine Learning Operations) is a set of practices that combines ML, DevOps, and Data Engineering to deploy, monitor, and maintain ML models in production reliably and efficiently. Goal: bridge the gap between ML experimentation and production systems.

16.2 ML Lifecycle

Stage	Description	Tools
Data Management	Collect, version, validate data	DVC, Great Expectations
Experimentation	Train & compare models	MLflow, Weights & Biases
Model Registry	Store & version trained models	MLflow Registry, Hugging Face Hub
Deployment	Serve model to production	Flask, FastAPI, BentoML, TorchServe
Monitoring	Track performance drift	Prometheus, Grafana, EvidentlyAI
Retraining	Automate retraining pipelines	Airflow, Kubeflow, SageMaker

16.3 Model Serving Approaches

- REST API (Flask/FastAPI): model wrapped as HTTP endpoint — client sends request, gets prediction
- Batch Inference: run predictions on large dataset offline (not real-time)
- Edge Deployment: deploy model on device (phone, IoT) — TensorFlow Lite, ONNX
- Cloud Deployment: AWS SageMaker, Google Vertex AI, Azure ML
- Containerisation: Docker packages model + dependencies — reproducible, portable
- Orchestration: Kubernetes manages containers at scale

16.4 Model Monitoring

Models degrade over time as real-world data distributions change:

Issue	Definition	Solution
Data Drift	Input feature distribution changes	Monitor feature statistics
Concept Drift	Relationship between X and Y changes	Monitor prediction accuracy
Model Degradation	Accuracy drops in production	Trigger retraining pipeline

Issue	Definition	Solution
Latency / Load	Slow predictions under high traffic	Load balancing, auto-scaling

16.5 CI/CD for ML

- Continuous Integration (CI): automated testing of code and model validation on commits
- Continuous Delivery (CD): automated deployment of trained models to staging/production
- ML Pipelines: Kubeflow Pipelines, Apache Airflow, Metaflow
- Experiment Tracking: MLflow logs parameters, metrics, artifacts for each run

■ **EXAM TIP: MLOps = DevOps + Data Engineering + ML. Key concerns: drift detection, model versioning, automated retraining. Docker = containerise model. Kubernetes = orchestrate containers. FastAPI is most popular for serving ML models.**

Topic 17: AI Ethics, Security & Privacy

17.1 AI Ethics

AI Ethics is the study of moral principles and guidelines that govern the design, development, and deployment of AI systems to ensure they are beneficial, fair, and trustworthy.

Core Ethical Principles:

Principle	Definition	Example
Fairness	AI should not discriminate	Unbiased hiring algorithms
Transparency	AI decisions should be explainable	Explainable AI (XAI), LIME, SHAP
Accountability	Responsibility for AI actions	Audit trails, human oversight
Privacy	Protect personal data	GDPR compliance, differential privacy
Beneficence	AI should benefit humanity	Healthcare AI improving diagnostics
Non-Maleficence	AI should not cause harm	Safe autonomous vehicles
Autonomy	Respect human agency and decision-making	Human-in-the-loop systems

17.2 AI Bias

Bias in AI arises from biased training data, flawed algorithm design, or biased human feedback. Biased models can cause significant harm in high-stakes decisions.

- Data Bias: training data is not representative of population
- Algorithmic Bias: model design favours certain outcomes
- Historical Bias: reflects societal prejudices in historical data
- Measurement Bias: errors in how features or labels are collected
- Sampling Bias: non-representative sample used for training

Mitigation Strategies:

- Re-sampling: balance training data across groups
- Re-weighting: assign higher weight to under-represented groups
- Adversarial Debiasing: explicitly train to remove sensitive attribute information
- Fairness Metrics: Demographic Parity, Equal Opportunity, Equalised Odds

17.3 Explainable AI (XAI)

XAI makes black-box ML models interpretable. Critical for trust, debugging, regulatory compliance, and high-stakes domains (medicine, law, finance).

Method	Full Name	How It Works
LIME	Local Interpretable Model-Agnostic Explanations	Fits simple model around single prediction
SHAP	SHapley Additive exPlanations	Uses game theory to assign feature contribution to each prediction
Grad-CAM	Gradient-weighted Class Activation Mapping	Highlights image regions influencing CNN decision
Attention Maps	Transformer attention weights	Shows which tokens model attends to

17.4 Data Privacy

- GDPR (General Data Protection Regulation): EU regulation — right to explanation, right to erasure
- Differential Privacy: add calibrated noise to data/queries — protects individual privacy while preserving aggregate statistics
- Federated Learning: train model across decentralised devices without sharing raw data
- Data Anonymisation: remove personally identifiable information (PII)
- Data Minimisation: collect only data necessary for the stated purpose

17.5 AI Security Threats

Threat	Description	Defence
Adversarial Attacks	Tiny perturbations fool model	Adversarial training, input validation
Data Poisoning	Corrupt training data to degrade model	Robust training, data validation
Model Inversion	Reconstruct training data from model	Differential privacy
Membership Inference	Determine if sample was in training set	Differential privacy
Model Stealing	Clone model via API queries	Rate limiting, output perturbation

17.6 Responsible AI Frameworks

- UNESCO Recommendation on AI Ethics (2021)
- EU AI Act: risk-based regulatory framework for AI systems
- Google's AI Principles: Be socially beneficial, avoid harm, be accountable

- Microsoft Responsible AI: Fairness, Reliability, Privacy, Inclusiveness, Transparency, Accountability

■ **EXAM TIP: SHAP uses game theory (Shapley values) to explain individual predictions.**

Differential Privacy adds noise to protect individuals. Federated Learning: data never leaves device — privacy-preserving ML. GDPR is the key regulation — right to explanation is directly linked to XAI.

Topic ★: Quick Reference — Key Formulas & Facts

Essential Formulas

Formula	Topic
Bayes: $P(A B) = P(B A) \cdot P(A) / P(B)$	Probability
$MSE = \sum (y_i - \hat{y}_i)^2 / n$	Regression Loss
Entropy $H = -\sum p_i \cdot \log_2(p_i)$	Decision Trees
$Gini = 1 - \sum p_i^2$	Decision Trees
$Precision = TP / (TP + FP)$	Classification Metrics
$Recall = TP / (TP + FN)$	Classification Metrics
$F1 = 2 \cdot P \cdot R / (P + R)$	Classification Metrics
$w = w - \alpha \cdot \nabla L$	Gradient Descent
$TF-IDF = TF \times \log(N/df)$	NLP
$\text{sigmoid}(z) = 1 / (1 + e^{-z})$	Logistic Regression / NN
$IQR = Q3 - Q1$ (outlier: $< Q1 - 1.5 \cdot IQR$)	Statistics / EDA
$z = (x - \mu) / \sigma$ (outlier if $ z > 3$)	Standardisation
$r^2 \in [-1, 1]$ — Pearson correlation	EDA
$IoU = \text{Area}(A \cap B) / \text{Area}(A \cup B)$	Object Detection

Algorithm Quick Comparison

Algorithm	Type	Scaling Needed	Handles Non-Linear	Interpretable
Linear Regression	Regression	Yes	No	Yes
Logistic Regression	Classification	Yes	No (linear boundary)	Yes
Decision Tree	Both	No	Yes	Yes
Random Forest	Both	No	Yes	Partial
SVM	Both	Yes	Yes (kernel)	No
KNN	Both	Yes	Yes	No
Naive Bayes	Classification	No	No	Yes

Algorithm	Type	Scaling Needed	Handles Non-Linear	Interpretable
Neural Network	Both	Yes	Yes	No (Black-box)
K-Means	Clustering	Yes	No	Yes
PCA	Dim Reduction	Yes	No (linear)	Partial

End of Notes — Best of Luck for your NSCT Exam!